

Dynamic Implementation of Queue

```
#include <stdio.h>

#include <stdlib.h>

struct Node {

    int data;

    struct Node* next;

};

struct Node *front = NULL, *rear = NULL;

void enqueue(int value) {

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {

        printf("Memory allocation failed!\n");

        return;

    }

    newNode->data = value;

    newNode->next = NULL;

    if (rear == NULL) {

        front = rear = newNode;

    } else {

        rear->next = newNode;

        rear = newNode;

    }

    printf("Enqueued %d\n", value);

}

void dequeue() {

    if (front == NULL) {

        printf("Queue is Empty (Underflow)!\n");

        return;

    }

    struct Node* temp = front;

    printf("Dequeued %d\n", temp->data);

    front = front->next;

    if (front == NULL) {
```

```

        rear = NULL;

    }

    free(temp);
}

void display() {
    if (front == NULL) {
        printf("Queue is Empty!\n");
        return;
    }

    struct Node* temp = front;
    printf("Queue elements: ");

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    printf("\n");
}

int main() {
    int choice, value;

    while (1) {
        printf("\n--- Dynamic Queue Menu ---\n");

        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:

```

```

        dequeue();

        break;

    case 3:

        display();

        break;

    case 4:

        exit(0);

    default:

        printf("Invalid choice! Please try again.\n");

    }

}

return 0;

}

```

Static Implementation of Circular Queue

```

#include <stdio.h>

#include <stdlib.h>

#define MAX 5

int queue[MAX];

int front = -1, rear = -1;

void enqueue(int value) {

    if ((rear + 1) % MAX == front) {

        printf("Circular Queue is Full (Overflow)!\n");

    } else {

        if (front == -1) {

            front = 0;

        }

        rear = (rear + 1) % MAX;

        queue[rear] = value;

        printf("Enqueued %d\n", value);

    }

}

void dequeue() {

```

```

if (front == -1) {

    printf("Circular Queue is Empty (Underflow)!\n");

} else {

    printf("Dequeued %d\n", queue[front]);

    if (front == rear) {

        front = rear = -1;

    } else {

        front = (front + 1) % MAX;

    }

}

}

void display() {

    if (front == -1) {

        printf("Circular Queue is Empty!\n");

    } else {

        printf("Queue elements: ");

        int i = front;

        while (1) {

            printf("%d ", queue[i]);

            if (i == rear) break;

            i = (i + 1) % MAX;

        }

        printf("\n");

    }

}

int main() {

    int choice, value;

    while (1) {

        printf("\n--- Circular Queue Menu ---\n");

        printf("1. Enqueue\n");

        printf("2. Dequeue\n");

        printf("3. Display\n");

        printf("4. Exit\n");

        printf("Enter your choice: ");

```

```

scanf("%d", &choice);

switch (choice){

    case 1:

        printf("Enter value to enqueue: ");

        scanf("%d", &value);

        enqueue(value);

        break;

    case 2:

        dequeue();

        break;

    case 3:

        display();

        break;

    case 4:

        exit(0);

    default:

        printf("Invalid choice! Please try again.\n");

}

}

return 0;

}

```

Static Implementation of Queue

```

#include <stdio.h>

#include <stdlib.h>

#define MAX 5

int queue[MAX];

int front = -1, rear = -1;

void enqueue(int value) {

    if (rear == MAX - 1) {

        printf("Queue is Full (Overflow)!\n");

    } else {

        if (front == -1) {

            front = 0;

        }
    }
}

```

```

    }

    rear++;

    queue[rear] = value;

    printf("Enqueued %d\n", value);

}

}

void dequeue() {

    if (front == -1 || front > rear) {

        printf("Queue is Empty (Underflow)!\n");

    } else {

        printf("Dequeued %d\n", queue[front]);

        front++;

        if (front > rear) {

            front = rear = -1;

        }

    }

}

void display() {

    if (front == -1 || front > rear) {

        printf("Queue is Empty!\n");

    } else {

        printf("Queue elements: ");

        for (int i = front; i <= rear; i++) {

            printf("%d ", queue[i]);

        }

        printf("\n");

    }

}

int main() {

    int choice, value;

    while (1) {

        printf("\n--- Static Queue Menu ---\n");

        printf("1. Enqueue\n");

        printf("2. Dequeue\n");

```

```
printf("3. Display\n");

printf("4. Exit\n");

printf("Enter your choice: ");

scanf("%d", &choice);

switch (choice) {

    case 1:

        printf("Enter value to enqueue: ");

        scanf("%d", &value);

        enqueue(value);

        break;

    case 2:

        dequeue();

        break;

    case 3:

        display();

        break;

    case 4:

        exit(0);

    default:

        printf("Invalid choice! Please try again.\n");

}

}

return 0;

}
```